

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	CH.POORNA TEJA	Reg. No.:	192472149
Section / Batch:	2024	Date:	22-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 AT3-COMPARATIVE ANALYSIS

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Floyd's Algorithm vs Warshall's
Algorithm (DP Comparison)
100 marks

1/1 answered

Q1 Floyd's Algorithm vs Warshall's Algorithm (DP Comparison)

100 marks

Given a graph, compute all-pairs shortest paths using Floyd's algorithm and transitive closure using Warshall's algorithm. Compare objectives and outputs. Analyze similarities in DP formulation. Evaluate time complexity. Discuss scalability. Generate insights on graph-based DP algorithms.

Your Code

```
1 n = int(input())
2
3 graph = []
4
5 for i in range(n):
6     graph.append(list(map(int, input().split())))
7
8 # Floyd's Algorithm - All Pairs Shortest Path
9 floyd = [row[:] for row in graph]
10
11 for k in range(n):
12     for i in range(n):
13         for j in range(n):
14             floyd[i][j] = min(floyd[i][j],
15                               floyd[i][k] + floyd[k][j])
16
17 # Warshall's Algorithm - Transitive Closure
18 warshall = [[0] * n for _ in range(n)]
19
20 for i in range(n):
21     for j in range(n):
22         if graph[i][j] != 0:
23             warshall[i][j] = 1
24
25 for i in range(n):
26     warshall[i][i] = 1
27
28 for k in range(n):
29     for i in range(n):
30         for j in range(n):
31             warshall[i][j] = warshall[i][j] or (
32                 warshall[i][k] and warshall[k][j]
33             )
34
35 print("Floyd's Algorithm:")
36 for row in floyd:
37     print(*row)
38
39 print("Warshall's Algorithm:")
40 for row in warshall:
41     print(*row)
42
43 print("Time Complexity: O(n^3)")
```

Submitted

Input

```
4
0 3 0 7
3 0 2 0
0 2 0 1
```

Output

```
Floyd's Algorithm:
0 1 0 1
1 0 1 0
1 0 1 0
1 0 1 0
Warshall's Algorithm:
```

Algorithm: Floyd's Algorithm vs Warshall's Algorithm in Python

1. Floyd's Algorithm – All-Pairs Shortest Path

Input: Weighted graph represented by an adjacency matrix.

Steps:

1. Read the number of vertices n.

2. Read the graph as an $n \times n$ matrix.
3. Copy the graph matrix into floyd.
4. For each vertex k :
 - For each source vertex i :
 - For each destination vertex j :
 - Check whether the path through k is shorter.
 - Update:

$$\text{floyd}[i][j] = \min(\text{floyd}[i][j], \text{floyd}[i][k] + \text{floyd}[k][j])$$
5. Print the final floyd matrix.
6. The matrix contains the shortest distance between every pair of vertices.

2. Warshall's Algorithm – Transitive Closure

Input: Graph represented by an adjacency matrix.

Steps:

1. Create an $n \times n$ matrix warshall filled with 0.
2. For every pair of vertices i and j :
 - If there is an edge from i to j , set $\text{warshall}[i][j] = 1$.
3. Set $\text{warshall}[i][i] = 1$ for every vertex.
4. For each intermediate vertex k :
 - For each source vertex i :
 - For each destination vertex j :
 - Check whether i can reach j through k .
 - Update:

$$\text{warshall}[i][j] = \text{warshall}[i][j] \text{ or } (\text{warshall}[i][k] \text{ and } \text{warshall}[k][j])$$
5. Print the final warshall matrix.
6. The matrix shows whether a path exists between every pair of vertices.

3. Complexity

- **Floyd's Algorithm:** $O(n^3)$
- **Warshall's Algorithm:** $O(n^3)$
- **Space Complexity:** $O(n^2)$ for both algorithms.

Main difference:

- **Floyd:** Finds the **shortest distance** between every pair of vertices.
- **Warshall:** Finds whether a **path exists** between every pair of vertices.